

Exercicios_PyTorch_Lista_3_RESOLVIDO

July 28, 2026

1 LISTA DE EXERCÍCIOS 3: PyTorch - Treinamento de Modelos

1.1 Datasets, DataLoaders e Ciclo de Treinamento (Backpropagation)

Dificuldade: Intermediária **Tempo estimado:** 60-90 minutos **Exercícios:** 10 (Níveis 1-5)

1.1.1 Pré-requisito

Complete as **LISTAS DE EXERCÍCIOS 1 e 2** antes de começar

1.1.2 Objetivos

- Criar Datasets customizados e DataLoaders
- Entender o ciclo completo de treinamento (forward, loss, backward, step)
- Compreender como o backpropagation calcula gradientes
- Montar um loop de treinamento multi-épocas
- Saber enviar modelo e dados para a GPU

1.1.3 Próximo

Após completar esta lista, você já terá o ciclo completo: dados → modelo → treinamento!

1.2 Setup

```
[ ]: import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import numpy as np

torch.manual_seed(42)
np.random.seed(42)

print(f'PyTorch versão: {torch.__version__}')
print(' Pronto para começar!')
```

PyTorch versão: 2.11.0+cpu

Pronto para começar!

2 NÍVEL 1: Datasets e DataLoaders

Preparando os dados antes de treinar qualquer modelo

2.1 EXERCÍCIO 1.1: Dataset customizado

Tarefa: 1. Crie dados sintéticos: - X com shape (200, 4), valores aleatórios (`torch.randn`) - y com shape (200,), calculado como $y = 2 \cdot X[:,0] - X[:,1] + 0.5 \cdot X[:,2] + \text{ruído}$ 2. Crie uma classe `MeuDataset(Dataset)` com `__init__`, `__len__` e `__getitem__` 3. Instancie o dataset e imprima o total de amostras 4. Acesse a amostra de índice 10 e imprima features e label

```
[ ]: # ===== EXERCÍCIO 1.1 =====
N_AMOSTRAS, N_FEATURES = 200, 4

# 1) Dados sintéticos.
#   Coeficientes VERDADEIROS: w = [2.0, -1.0, 0.5, 0.0] / b = 0.0
#   sigma do ruído = 0.1 -> piso teórico do MSE ~ 0.1**2 = 0.01
X = torch.randn(N_AMOSTRAS, N_FEATURES)
ruído = 0.1 * torch.randn(N_AMOSTRAS)
y = 2 * X[:, 0] - X[:, 1] + 0.5 * X[:, 2] + ruído

# 2) Dataset customizado (map-style): contrato = __init__ / __len__ /
↳ __getitem__
class MeuDataset(Dataset):
    """Dataset in-memory. Mapeia um índice inteiro -> (features, label)."""

    def __init__(self, X: torch.Tensor, y: torch.Tensor) -> None:
        if X.shape[0] != y.shape[0]:
            raise ValueError(f"X e y incompatíveis: {X.shape[0]} vs {y.
↳ shape[0]}")
        # .float() explícito: nn.Linear opera em float32. Se os dados viessem de
        # np.float64 ou de torch.randint (int64), o forward quebraria por dtype.
        self.X = X.float()
        self.y = y.float()

    def __len__(self) -> int:
        return self.X.shape[0]

    def __getitem__(self, idx: int) -> tuple[torch.Tensor, torch.Tensor]:
        # Retorna VIEWS do tensor original (não cópias). O DataLoader empilha os
        # itens com torch.stack, e é aí que a memória do batch é alocada.
        return self.X[idx], self.y[idx]
```

```

# 3) Instanciação
dataset = MeuDataset(X, y)
print(f"Total de amostras: {len(dataset)}")

# 4) Amostra de índice 10
features_10, label_10 = dataset[10]
print(f"\nAmostra[10] features: {features_10} shape={tuple(features_10.
    ↪shape)}")
print(f"Amostra[10] label : {label_10.item():.4f} shape={tuple(label_10.
    ↪shape)}")

# Sanity check: o label bate com a equação geradora (a menos do ruído)?
esperado = 2 * features_10[0] - features_10[1] + 0.5 * features_10[2]
print(f"\nSem ruído seria: {esperado.item():.4f} | resíduo: {(label_10 -
    ↪esperado).item():+.4f}")

```

Total de amostras: 200

Amostra[10] features: tensor([-1.5576, 0.9956, -0.8798, -0.6011]) shape=(4,)
 Amostra[10] label : -4.5175 shape=()

Sem ruído seria: -4.5507 | resíduo: +0.0332

2.1.1 Nota — Dataset map-style vs. iterable-style

O `torch.utils.data.Dataset` implementado acima é **map-style**: o `DataLoader` sorteia índices e chama `__getitem__(i)`. É o que permite `shuffle=True`, `random_split()` e `Subset`.

Para dados que não cabem em memória ou chegam em stream (Kafka, arquivos gigantes), usa-se `IterableDataset`, que implementa `__iter__` — mas perde shuffling global e exige cuidado manual com `num_workers` para não duplicar amostras.

Regra prática: se `len()` é conhecido e o acesso aleatório é barato, use map-style.

2.2 EXERCÍCIO 1.2: DataLoader

Tarefa: 1. Crie um `DataLoader` a partir do dataset do exercício anterior, com `batch_size=16` e `shuffle=True` 2. Imprima o número total de batches (`len(dataloader)`) 3. Pegue o primeiro batch e imprima os shapes de `X_batch` e `y_batch`

```

[ ]: # ===== EXERCÍCIO 1.2 =====
BATCH_SIZE = 16

dataloader = DataLoader(
    dataset,
    batch_size=BATCH_SIZE,
    shuffle=True,      # reembaralha a cada época -> quebra correlação de ordem
    drop_last=False,   # default: mantém o último batch incompleto
)

```

```
# 2) Número de batches = ceil(200 / 16) = 13 (12 cheios + 1 com 8 amostras)
print(f"len(dataset)      = {len(dataset)}")
print(f"len(dataloader) = {len(dataloader)} -> ceil({len(dataset)})/
    ↳{BATCH_SIZE}")

# 3) Primeiro batch. next(iter(...)) cria um iterador descartável só para
    ↳inspeção.
X_batch, y_batch = next(iter(dataloader))
print(f"\nX_batch: shape={tuple(X_batch.shape)} dtype={X_batch.dtype}")
print(f"y_batch: shape={tuple(y_batch.shape)} dtype={y_batch.dtype}")
print(f"\n y_batch é 1-D (16,) e a saída do nn.Linear será 2-D (16, 1) -> ver
    ↳Ex. 2.1")
```

```
len(dataset)      = 200
len(dataloader) = 13 -> ceil(200/16)
```

```
X_batch: shape=(16, 4) dtype=torch.float32
y_batch: shape=(16,) dtype=torch.float32
```

y_batch é 1-D (16,) e a saída do nn.Linear será 2-D (16, 1) -> ver Ex. 2.1

2.2.1 Nota — parâmetros do DataLoader que importam

Parâmetro	Efeito	Quando mexer
batch_size	Amostras por passo de gradiente	Trade-off memória × ruído do gradiente
shuffle	Reembaralha índices a cada época	True no treino, False na validação/teste
drop_last	Descarta o último batch incompleto	True quando BatchNorm sofre com batch pequeno
num_workers	Processos de carregamento em paralelo	>0 quando o I/O é o gargalo (não neste caso)
pin_memory	Memória page-locked p/ transferência à GPU	Só faz sentido com CUDA

`len(dataloader) = ceil(len(dataset) / batch_size)` com `drop_last=False`; `floor(...)` com `drop_last=True`.

2.3 EXERCÍCIO 1.3: Iterando por uma época completa

Tarefa: 1. Usando o mesmo `DataLoader`, itere por TODOS os batches (isso é uma **época**) 2. Para cada batch, imprima o índice do batch e o shape de `X_batch` 3. Ao final, imprima quantos batches foram processados no total 4. Esse total é igual ao valor de `len(dataloader)`?

```
[ ]: # ===== EXERCÍCIO 1.3 =====
total_batches = 0
total_amstras = 0

for i, (X_batch, y_batch) in enumerate(dataloader):
    print(f"batch {i:2d} | X_batch shape = {tuple(X_batch.shape)}")
    total_batches += 1
    total_amstras += X_batch.shape[0]

print(f"\nBatches processados : {total_batches}")
print(f"len(dataloader)       : {len(dataloader)}")
print(f"Iguais?                : {total_batches == len(dataloader)}")
print(f"Amstras vistas         : {total_amstras} (== len(dataset) = 200)
↪ {len(dataset)}")
```

```
batch  0 | X_batch shape = (16, 4)
batch  1 | X_batch shape = (16, 4)
batch  2 | X_batch shape = (16, 4)
batch  3 | X_batch shape = (16, 4)
batch  4 | X_batch shape = (16, 4)
batch  5 | X_batch shape = (16, 4)
batch  6 | X_batch shape = (16, 4)
batch  7 | X_batch shape = (16, 4)
batch  8 | X_batch shape = (16, 4)
batch  9 | X_batch shape = (16, 4)
batch 10 | X_batch shape = (16, 4)
batch 11 | X_batch shape = (16, 4)
batch 12 | X_batch shape = (8, 4)
```

```
Batches processados : 13
len(dataloader)      : 13
Iguais?              : True
Amstras vistas       : 200 (== len(dataset) = 200)
```

3 NÍVEL 2: Forward, Loss e Backward

As duas primeiras etapas do ciclo de treinamento

3.1 EXERCÍCIO 2.1: Forward pass e cálculo da loss

Tarefa: 1. Crie um modelo simples: `modelo = nn.Linear(4, 1)` 2. Pegue um batch do `DataLoader` do Nível 1 3. Faça o forward pass: `saida = modelo(X_batch)` 4. Calcule a loss usando `nn.MSELoss()` entre `saida` e `y_batch` - Dica: pode ser necessário ajustar o shape de `y_batch` com `.view(-1, 1)` ou `.unsqueeze(1)` 5. Imprima o valor da loss com `.item()`

```
[ ]: # ===== EXERCÍCIO 2.1 =====
import warnings

# 1) Modelo linear: 4 features -> 1 saída. Equivale a  $y_{\hat{}} = X @ W.T + b$ 
modelo = nn.Linear(in_features=4, out_features=1)
print(f"weight shape: {tuple(modelo.weight.shape)} | bias shape: {tuple(modelo.
    ↳bias.shape)}")

# 2) Um batch
X_batch, y_batch = next(iter(dataloader))

# 3) Forward pass
saida = modelo(X_batch)
print(f"\nsaida : {tuple(saida.shape)}")
print(f"y_batch: {tuple(y_batch.shape)}")

criterio = nn.MSELoss()

# --- GOTCHA: (16,1) vs (16,) faz BROADCASTING para (16,16) ---
with warnings.catch_warnings(record=True) as w:
    warnings.simplefilter("always")
    loss_errada = criterio(saida, y_batch)
    print(f"\n[ERRADO] loss = {loss_errada.item():.4f}")
    if w:
        print(f"[ERRADO] warning: {str(w[0].message).splitlines()[0]}")

# 4) Forma correta: alinhar os shapes
loss = criterio(saida, y_batch.view(-1, 1))
print(f"\n[CORRETO] loss = {loss.item():.4f}")

# 5) .item() extrai o escalar Python e DESCONECTA do grafo (não usar dentro do
    ↳backward)
print(f"tipo de loss: {type(loss)} | grad_fn: {loss.grad_fn}")
```

weight shape: (1, 4) | bias shape: (1,)

saida : (16, 1)
y_batch: (16,)

[ERRADO] loss = 3.8860

[ERRADO] warning: Using a target size (torch.Size([16])) that is different to the input size (torch.Size([16, 1])). This will likely lead to incorrect results due to broadcasting. Please ensure they have the same size.

[CORRETO] loss = 3.4953

tipo de loss: <class 'torch.Tensor'> | grad_fn: <MseLossBackward0 object at 0x78628a7c46d0>

3.2 EXERCÍCIO 2.2: Backward pass - explorando gradientes

Tarefa: 1. Antes de chamar `.backward()`, imprima `modelo.weight.grad` (deve ser `None`) 2. Chame `loss.backward()` 3. Imprima `modelo.weight.grad` e `modelo.bias.grad` 4. Em uma célula markdown ou print, explique com suas palavras o que esses valores representam

```
[ ]: # ===== EXERCÍCIO 2.2 =====
# 1) Antes do backward: .grad ainda não existe
print(f"modelo.weight.grad ANTES : {modelo.weight.grad}")
print(f"modelo.bias.grad   ANTES : {modelo.bias.grad}")

# 2) Backward: percorre o DAG de trás para frente aplicando a regra da cadeia
loss.backward()

# 3) Depois do backward
print(f"\nmodelo.weight.grad DEPOIS: {modelo.weight.grad}")
print(f"shape: {tuple(modelo.weight.grad.shape)} (idêntico ao do próprio weight)")
print(f"\nmodelo.bias.grad   DEPOIS: {modelo.bias.grad}")

# 4) Validação analítica.
#   MSE = (1/N) * sum((ŷ - y)²)  ->  dL/dW = (2/N) * (ŷ - y)ᵀ @ X
#                                   dL/db = (2/N) * sum(ŷ - y)
with torch.no_grad():
    N = X_batch.shape[0]
    residuo = modelo(X_batch) - y_batch.view(-1, 1)      # (16,1)
    grad_w_manual = (2.0 / N) * (residuo.T @ X_batch)      # (1,4)
    grad_b_manual = (2.0 / N) * residuo.sum()

print(f"\ngrad_w manual : {grad_w_manual}")
print(f"bate com autograd? {torch.allclose(grad_w_manual, modelo.weight.grad, atol=1e-6)}")
print(f"grad_b manual : {grad_b_manual.item():.6f} | autograd: {modelo.bias.grad.item():.6f}")
```

```
modelo.weight.grad ANTES : None
modelo.bias.grad   ANTES : None
```

```
modelo.weight.grad DEPOIS: tensor([[ -3.2347,  0.6717, -1.4206, -0.3485]])
shape: (1, 4) (idêntico ao do próprio weight)
```

```
modelo.bias.grad   DEPOIS: tensor([2.1086])
```

```
grad_w manual : tensor([[ -3.2347,  0.6717, -1.4206, -0.3485]])
bate com autograd? True
grad_b manual : 2.108557 | autograd: 2.108557
```

3.2.1 Resposta — o que esses gradientes representam

`modelo.weight.grad[0, j]` é a derivada parcial $\text{loss} / \mathbf{w}_j$ avaliada no ponto atual dos parâmetros, para *este* batch específico. Em termos operacionais:

- **Sinal** — indica a direção em que o peso aumentaria a loss. O otimizador anda no sentido *oposto* ($\mathbf{w} \leftarrow \mathbf{w} - \text{lr} \cdot \mathbf{w}$).
- **Magnitude** — sensibilidade local da loss àquele peso. Gradiente grande = o peso está longe do valor que minimiza o erro (ou a feature tem escala grande, daí a importância de normalizar entradas).
- **None antes do backward()** — o buffer `.grad` só é alocado quando o autograd efetivamente propaga algo até o tensor folha. Não é zero: é *inexistente*.
- **Acúmulo** — cada `backward()` **soma** em `.grad` em vez de sobrescrever. Isso é intencional (viabiliza *gradient accumulation* para simular batches grandes), e é exatamente por isso que `zero_grad()` é obrigatório a cada iteração.

A verificação analítica acima confirma que o autograd está computando a fórmula fechada do gradiente do MSE — o motor não é uma caixa-preta, é a regra da cadeia aplicada sobre o DAG.

4 NÍVEL 3: Optimizer e Loop de Treinamento

Fechando o ciclo: forward $\rightarrow$ loss $\rightarrow$ backward $\rightarrow$ step

4.1 EXERCÍCIO 3.1: zero_grad e step

Tarefa: 1. Crie um otimizador: `otimizador = optim.SGD(modelo.parameters(), lr=0.01)`
2. Guarde uma cópia dos pesos atuais: `pesos_antes = modelo.weight.clone()` 3. Repita o forward + loss + backward do exercício 2.1/2.2 4. Chame `otimizador.step()` e depois `otimizador.zero_grad()` 5. Compare `pesos_antes` com `modelo.weight` depois do `step()`. Eles mudaram?

```
[ ]: # ===== EXERCÍCIO 3.1 =====
otimizador = optim.SGD(modelo.parameters(), lr=0.01)

# 2) Cópia dos pesos. .detach().clone() é o padrão seguro:
#   - clone() sozinho mantém grad_fn (vira nó do grafo);
#   - detach() sozinho COMPARTILHA storage -> seria mutado por operações
#     ↪ in-place.
pesos_antes = modelo.weight.detach().clone()

# O Ex. 2.2 já deixou gradientes acumulados em .grad. Se chamarmos backward()
# de novo sem zerar, o PyTorch SOMA os gradientes (comportamento by design,
# útil para gradient accumulation). Zeramos ANTES para isolar o efeito do
# ↪ step.
otimizador.zero_grad()

# 3) forward -> loss -> backward
saida = modelo(X_batch)
```



```

loss = critério(saida, y_batch.view(-1, 1))
loss.backward()

grad_usado = modelo.weight.grad.detach().clone()

# 4) step() aplica a regra de atualização; zero_grad() limpa o buffer
otimizador.step()
otimizador.zero_grad()

# 5) Comparação
print(f"pesos ANTES : {pesos_antes}")
print(f"pesos DEPOIS : {modelo.weight.detach()}")
print(f"delta : {modelo.weight.detach() - pesos_antes}")
print(f"\nMudaram? {not torch.equal(pesos_antes, modelo.weight.detach())}")

# Validação da regra do SGD puro:  $w_{novo} = w_{antigo} - lr * grad$ 
esperado = pesos_antes - 0.01 * grad_usado
print(f"w_novo == w_antigo - lr*grad ? {torch.allclose(esperado, modelo.weight.
↳detach(), atol=1e-7)}")

```

```

pesos ANTES : tensor([[ 0.2785, -0.0747,  0.2124, -0.2935]])
pesos DEPOIS : tensor([[ 0.3108, -0.0814,  0.2266, -0.2900]])
delta : tensor([[ 0.0323, -0.0067,  0.0142,  0.0035]])

```

Mudaram? True

w_novo == w_antigo - lr*grad ? True

4.1.1 Nota — ordem de zero_grad() e step()

O enunciado pede `step()` → `zero_grad()`. Funciona, mas o padrão canônico é zerar no início da iteração:

```

for X_batch, y_batch in dataloader:
    otimizador.zero_grad() # ← início
    loss = critério(modelo(X_batch), y_batch.view(-1, 1))
    loss.backward()
    otimizador.step()

```

As duas ordens são equivalentes desde que nenhum `backward()` ocorra entre o `step()` e o `zero_grad()`. Zerar no início é mais robusto porque não depende de nada ter sido limpo por uma iteração anterior — e evita exatamente o bug que apareceu neste exercício (gradiente residual do Ex. 2.2).

Nunca chame `zero_grad()` entre `backward()` e `step()`: o `step()` leria gradientes zerados e o modelo não aprenderia nada, silenciosamente.

4.2 EXERCÍCIO 3.2: Loop de treinamento - 1 época

Tarefa: 1. Usando o `DataLoader`, o modelo e o otimizador já criados 2. Escreva um loop que passe por todos os batches de **UMA** época 3. Para cada batch, na ordem correta: `zero_grad` →

forward → loss → backward → step 4. Acumule a loss de cada batch e, ao final, imprima a loss média da época

```
[ ]: # ===== EXERCÍCIO 3.2 =====
modelo.train() # sem efeito prático aqui (não há Dropout/BatchNorm), mas é o
↳ hábito correto

soma_losses = 0.0      # média simples por batch
soma_ponderada = 0.0   # média ponderada pelo n° de amostras
n_amostras = 0

for X_batch, y_batch in dataloader:
    # ORDEM CANÔNICA: zerar -> forward -> loss -> backward -> step
    otimizador.zero_grad()
    saida = modelo(X_batch)
    loss = criterio(saida, y_batch.view(-1, 1))
    loss.backward()
    otimizador.step()

    # .item() aqui é obrigatório: acumular o TENSOR loss manteria todo o grafo
    # de todas as iterações vivo na memória (vazamento clássico).
    soma_losses += loss.item()
    soma_ponderada += loss.item() * X_batch.shape[0]
    n_amostras += X_batch.shape[0]

media_simples = soma_losses / len(dataloader)
media_ponderada = soma_ponderada / n_amostras

print(f"Loss média (por batch)      : {media_simples:.6f}")
print(f"Loss média (por amostra)    : {media_ponderada:.6f}")
print("\nAs duas divergem porque o último batch tem 8 amostras, não 16.")
print("A média ponderada é a métrica correta quando drop_last=False.")
```

```
Loss média (por batch)      : 3.060625
Loss média (por amostra)    : 3.098810
```

As duas divergem porque o último batch tem 8 amostras, não 16.
A média ponderada é a métrica correta quando drop_last=False.

4.3 EXERCÍCIO 3.3: Loop de treinamento multi-épocas

Tarefa: 1. Reinicie o modelo e o otimizador (para começar do zero) 2. Repita o loop do exercício anterior, mas para **20 épocas** 3. Imprima a loss média a cada 5 épocas 4. A loss está diminuindo? O que isso indica sobre o aprendizado do modelo?

```
[ ]: # ===== EXERCÍCIO 3.3 =====
# 1) Reset completo: modelo novo -> otimizador novo.
```

```

# O otimizador guarda REFERÊNCIAS aos parâmetros; recriar o modelo sem
↳recriar
# o otimizador faria ele otimizar os pesos antigos (bug silencioso).
torch.manual_seed(42)
modelo = nn.Linear(4, 1)
otimizador = optim.SGD(modelo.parameters(), lr=0.01)
criterio = nn.MSELoss()

N_EPOCAS = 20
historico = []

for epoca in range(1, N_EPOCAS + 1):
    modelo.train()
    soma, n = 0.0, 0

    for X_batch, y_batch in dataloader:
        otimizador.zero_grad()
        saida = modelo(X_batch)
        loss = criterio(saida, y_batch.view(-1, 1))
        loss.backward()
        otimizador.step()

        soma += loss.item() * X_batch.shape[0]
        n += X_batch.shape[0]

    loss_epoca = soma / n
    historico.append(loss_epoca)

    # 3) Log a cada 5 épocas (e na primeira, para ter a baseline)
    if epoca == 1 or epoca % 5 == 0:
        print(f"Época {epoca:2d}/{N_EPOCAS} | loss = {loss_epoca:.6f}")

print(f"\nRedução total: {historico[0]:.4f} -> {historico[-1]:.4f} "
      f"({100 * (1 - historico[-1] / historico[0]):.1f}% de queda)")
print(f"Piso teórico (variância do ruído): {0.1**2:.4f}")

print("\nParâmetros aprendidos vs. verdadeiros:")
for i, (aprendido, verdadeiro) in enumerate(zip(modelo.weight.detach().view(-1).
↳tolist(),
                                                [2.0, -1.0, 0.5, 0.0])):
    print(f" w[{i}] = {aprendido:+.4f} (verdadeiro: {verdadeiro:+.1f})")
print(f" b = {modelo.bias.item():+.4f} (verdadeiro: +0.0)")

```

```

Época 1/20 | loss = 4.245111
Época 5/20 | loss = 0.498785
Época 10/20 | loss = 0.045706
Época 15/20 | loss = 0.012814

```

Época 20/20 | loss = 0.010291

Redução total: 4.2451 -> 0.0103 (99.8% de queda)

Piso teórico (variância do ruído): 0.0100

Parâmetros aprendidos vs. verdadeiros:

```
w[0] = +1.9980 (verdadeiro: +2.0)
w[1] = -0.9916 (verdadeiro: -1.0)
w[2] = +0.4973 (verdadeiro: +0.5)
w[3] = -0.0029 (verdadeiro: +0.0)
b     = -0.0127 (verdadeiro: +0.0)
```

4.3.1 Resposta — a loss está diminuindo? O que isso indica?

Sim, monotonicamente, e satura perto de **0.01**, que é exatamente a variância do ruído gaussiano injetado ($\sigma = 0.1 \rightarrow \sigma^2 = 0.01$). Esse é o **erro irredutível de Bayes** do problema: nenhum modelo, por mais expressivo, consegue baixar disso, porque o ruído é independente de X .

Interpretação em três camadas:

1. **A loss cai** $\rightarrow$ os gradientes apontam para uma direção de descida útil e o *learning rate* está numa faixa estável (não diverge, não estagna).
2. **Os pesos convergem para [2, -1, 0.5, 0]** $\rightarrow$ o modelo não só reduziu a loss, ele **recuperou o processo gerador dos dados**. É a evidência forte: loss baixa por si só pode ser overfitting, coeficientes corretos não.
3. **w[3] = 0** $\rightarrow$ o modelo aprendeu sozinho que a quarta feature é irrelevante, sem regularização explícita.

Ressalva metodológica: aqui só existe conjunto de **treino**. Loss de treino caindo indica capacidade de ajuste, **não** generalização. Num cenário real seria necessário `random_split()` e uma curva de validação para detectar overfitting.

5 NÍVEL 4: Fundamentos de GPU

Como enviar modelo e dados para a GPU (sem treinar na GPU ainda)

5.1 EXERCÍCIO 4.1: Enviando modelo e dados para a GPU

Tarefa: 1. Crie `device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')` 2. Envie o modelo para o device: `modelo = modelo.to(device)` 3. Dentro de um loop pelo `DataLoader`, envie cada batch (`X_batch`, `y_batch`) para o device 4. Imprima o `.device` de um tensor **antes** e **depois** de chamar `.to(device)`

Nota: mesmo sem GPU disponível, o código deve funcionar normalmente (cai para 'cpu')

```
[ ]: # ===== EXERCÍCIO 4.1 =====
# 1) Device-agnostic code: o mesmo script roda em máquina com e sem GPU
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

```

print(f"CUDA disponível: {torch.cuda.is_available()}")
print(f"Device escolhido: {device}")
if device.type == 'cuda':
    print(f"GPU: {torch.cuda.get_device_name(0)}")

# 2) Modelo -> device. .to() em nn.Module é IN-PLACE (move os parâmetros e
↳ retorna
# self). Em Tensor, .to() NÃO é in-place: retorna uma cópia no device
↳ destino.
modelo = modelo.to(device)
print(f"\nParâmetros do modelo em: {next(modelo.parameters()).device}")

# 3) + 4) Batches -> device
for i, (X_batch, y_batch) in enumerate(dataloader):
    if i == 0:
        print(f"\nANTES do .to(device) -> X_batch.device = {X_batch.device}")

        X_batch = X_batch.to(device)          # reatribuição obrigatória
        y_batch = y_batch.to(device)

        if i == 0:
            print(f"DEPOIS do .to(device) -> X_batch.device = {X_batch.device}")

        # Regra de ouro: modelo e entrada precisam estar no MESMO device,
        # caso contrário -> RuntimeError: Expected all tensors to be on the same
↳ device
        _ = modelo(X_batch)

print(f"\n {i + 1} batches transferidos e processados em '{device}'.")

```

CUDA disponível: False

Device escolhido: cpu

Parâmetros do modelo em: cpu

ANTES do .to(device) -> X_batch.device = cpu

DEPOIS do .to(device) -> X_batch.device = cpu

13 batches transferidos e processados em 'cpu'.

5.1.1 Nota — semântica de .to()

Objeto	.to(device) é in-place?	Precisa reatribuir?
nn.Module	Sim (move parâmetros e buffers, retorna self)	Não, mas <code>modelo = modelo.to(device)</code> é idiomático

Objeto	.to(device) é in-place?	Precisa reatribuir?
<code>torch.Tensor</code>	Não (retorna nova cópia no device destino)	Sim , obrigatoriamente

Consequência prática: esquecer o `X_batch =` na frente do `.to(device)` é um erro silencioso — o tensor original permanece na CPU e o forward estoura com `RuntimeError: Expected all tensors to be on the same device`.

Complementos para o pipeline em GPU: - `pin_memory=True` no `DataLoader` + `.to(device, non_blocking=True)` → transferência H2D assíncrona, sobreposta ao cálculo. - `non_blocking=True` sem `pin_memory` não tem efeito: memória paginável força cópia síncrona.

6 NÍVEL 5: Desafio Integrado

Junte tudo o que aprendeu nesta lista

6.1 DESAFIO 5.1: Pipeline completo de treinamento

Tarefa: 1. Combine tudo: - `Dataset` customizado (pode reaproveitar o do Nível 1) - `DataLoader` - Modelo (`nn.Linear`) - `device` (envie modelo e dados) - Loop de treinamento com múltiplas épocas (ex: 30) 2. Ao final, imprima a loss da **primeira** época e a loss da **última** época 3. Crie um dado novo (fora do dataset de treino) e faça uma predição com o modelo treinado 4. Imprima o resultado da predição

```
[ ]: # ===== DESAFIO 5.1: PIPELINE COMPLETO =====
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader

# ----- Configuração -----
SEED = 42
N_AMOSTRAS, N_FEATURES = 200, 4
BATCH_SIZE = 16
N_EPOCAS = 30
LR = 0.01
COEF_VERDADEIROS = [2.0, -1.0, 0.5, 0.0]

torch.manual_seed(SEED)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Device: {device}\n")

# ----- 1. Dados -----
X = torch.randn(N_AMOSTRAS, N_FEATURES)
y = 2 * X[:, 0] - X[:, 1] + 0.5 * X[:, 2] + 0.1 * torch.randn(N_AMOSTRAS)
```

```

class MeuDataset(Dataset):
    def __init__(self, X: torch.Tensor, y: torch.Tensor) -> None:
        if X.shape[0] != y.shape[0]:
            raise ValueError("X e y com número de amostras diferente")
        self.X, self.y = X.float(), y.float()

    def __len__(self) -> int:
        return self.X.shape[0]

    def __getitem__(self, idx: int) -> tuple[torch.Tensor, torch.Tensor]:
        return self.X[idx], self.y[idx]

dataset = MeuDataset(X, y)
dataloader = DataLoader(
    dataset,
    batch_size=BATCH_SIZE,
    shuffle=True,
    # pin_memory só ajuda quando há GPU: aloca memória page-locked e permite
    # transferência assíncrona com non_blocking=True.
    pin_memory=(device.type == 'cuda'),
)

# ----- 2. Modelo, loss, otimizador -----
modelo = nn.Linear(N_FEATURES, 1).to(device)
criterio = nn.MSELoss()
otimizador = optim.SGD(modelo.parameters(), lr=LR)

# ----- 3. Loop de treinamento -----
historico: list[float] = []

for epoca in range(1, N_EPOCAS + 1):
    modelo.train()
    soma, n = 0.0, 0

    for X_batch, y_batch in dataloader:
        X_batch = X_batch.to(device, non_blocking=True)
        y_batch = y_batch.to(device, non_blocking=True)

        otimizador.zero_grad(set_to_none=True) # default no PyTorch >= 2.0:
        ↪ mais barato que zerar
        saida = modelo(X_batch)
        loss = criterio(saida, y_batch.view(-1, 1))
        loss.backward()
        otimizador.step()

```

```

        soma += loss.item() * X_batch.shape[0]
        n += X_batch.shape[0]

    historico.append(soma / n)
    if epoca == 1 or epoca % 10 == 0:
        print(f"Época {epoca:2d}/{N_EPOCAS} | loss = {historico[-1]:.6f}")

# ----- 4. Resultado do treinamento -----
print("\n" + "=" * 52)
print(f"Loss da PRIMEIRA época : {historico[0]:.6f}")
print(f"Loss da ÚLTIMA época   : {historico[-1]:.6f}")
print(f"Fator de redução       : {historico[0] / historico[-1]:.1f}x")
print("=" * 52)

# ----- 5. Inferência em dado novo -----
modelo.eval() # desliga Dropout/BatchNorm de treino (inócuo aqui, obrigatório
↳ em redes reais)

x_novo = torch.tensor([[1.0, -2.0, 3.0, 0.5]])
alvo_analitico = sum(c * v for c, v in zip(COEF_VERDADEIROS, x_novo.view(-1).
↳ tolist()))

# inference_mode > no_grad: além de não gravar o grafo, desliga o version
↳ counter
# dos tensores -> menos overhead. Só não use se for precisar do resultado em
↳ autograd.
with torch.inference_mode():
    predicao = modelo(x_novo.to(device))

print(f"\nEntrada nova      : {x_novo.view(-1).tolist()}")
print(f"Predição           : {predicao.item():.4f}")
print(f"Valor teórico       : {alvo_analitico:.4f} (2*1 - (-2) + 0.5*3 + 0*0.5)")
print(f"Erro absoluto       : {abs(predicao.item() - alvo_analitico):.4f}")

print("\nParâmetros recuperados:")
for i, w in enumerate(modelo.weight.detach().cpu().view(-1).tolist()):
    print(f"  w[{i}] = {w:+.4f} (verdadeiro: {COEF_VERDADEIROS[i]:+.1f})")
print(f"  b      = {modelo.bias.item():+.4f} (verdadeiro: +0.0)")

```

Device: cpu

```

Época  1/30 | loss = 5.811058
Época 10/30 | loss = 0.060613
Época 20/30 | loss = 0.010320
Época 30/30 | loss = 0.010086

```



```

=====
Loss da PRIMEIRA época : 5.811058
Loss da ÚLTIMA época   : 0.010086
Fator de redução       : 576.2x
=====

Entrada nova          : [1.0, -2.0, 3.0, 0.5]
Predição             : 5.5021
Valor teórico         : 5.5000  (2*1 - (-2) + 0.5*3 + 0*0.5)
Erro absoluto         : 0.0021

Parâmetros recuperados:
w[0] = +2.0012  (verdadeiro: +2.0)
w[1] = -0.9991  (verdadeiro: -1.0)
w[2] = +0.5042  (verdadeiro: +0.5)
w[3] = -0.0067  (verdadeiro: +0.0)
b     = -0.0065  (verdadeiro: +0.0)

```